Lesson 10 Teacher's Key: Play It Again

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Splitting a scene object's work into named, reusable methods (ClearEnemies and SpawnEnemies) and reaching it from GameManager with an Inspector reference - the same drag-a-reference-in-the-Inspector move students already used for the enemy prefab in Lesson 5.

Level 1 - Full Worked Solution

- GameManager gets a public EnemySpawner spawner field, filled by dragging the spawner object onto the Inspector slot - exactly how the enemy prefab slot was filled in Lesson 5. ResetGame() is an ordinary (non-static) method for that reason: Inspector slots belong to one component instance, so a static method could not use one. ResetGame() is only ever called from GameManager's own Update(), which is an instance method, so nothing else changes.
- Splitting the old inline spawn code into ClearEnemies() and SpawnEnemies() as two separate public methods mirrors the same 'name the reusable pieces' idea as Godot's start_game(), just split into two calls instead of one.
- GameManager.ResetGame() orchestrates both: clear, reset variables, respawn - the actual 'reset the game' logic lives here, one level up from EnemySpawner's more mechanical spawn/clear methods.

Highlighted lines are new this week; everything else is code you already have.

```
// EnemySpawner.cs (complete):
public GameObject enemyPrefab;
public int enemyCount = 5;

void Start() {
    SpawnEnemies();
}

public void ClearEnemies() {
    foreach (GameObject oldEnemy in
GameObject.FindGameObjectsWithTag("Enemy")) {
        Destroy(oldEnemy);
    }
}

public void SpawnEnemies() {
    for (int enemyIndex = 0; enemyIndex < enemyCount; enemyIndex++) {
```

```
        Vector3 pos = new Vector3(-4f + enemyIndex * 2f, 4f, 0f);
        Instantiate(enemyPrefab, pos, Quaternion.identity);
    }
}

// GameManager.cs (the new and revised members):
using UnityEngine.InputSystem;

public static bool gameActive = true;

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) {
        hasWon = true;
        gameActive = false;
        AudioSource.PlayClipAtPoint(instance.winSound,
Camera.main.transform.position);  // your Lesson 6 Level 3 sound
    }
}

public static void RegisterEscape() {
    lives--;
    if (lives <= 0) {
        hasLost = true;
        gameActive = false;
        AudioSource.PlayClipAtPoint(instance.gameOverSound,
Camera.main.transform.position);  // your Lesson 8 Level 3 sound
    } else {
        AudioSource.PlayClipAtPoint(instance.lifeLostSound,
Camera.main.transform.position);  // your Lesson 8 Level 3 sound
    }
}

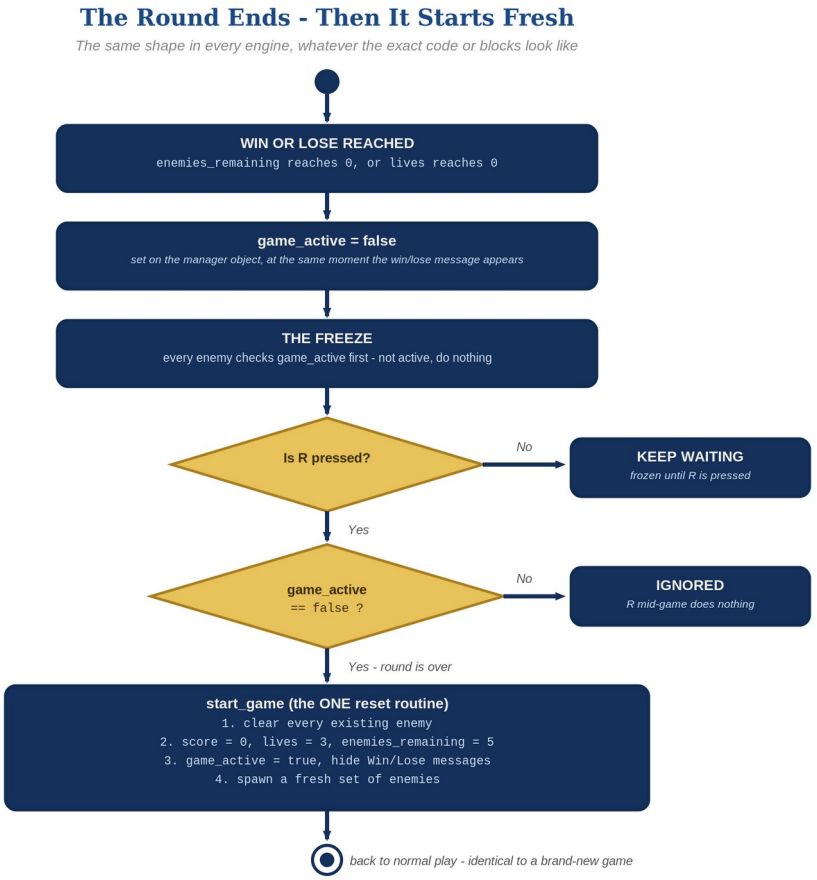
public EnemySpawner spawner;

public void ResetGame() {
    spawner.ClearEnemies();
    score = 0;
    lives = 3;
    enemiesRemaining = 5;
    hasWon = false;
    hasLost = false;
    gameActive = true;
    spawner.SpawnEnemies();
}

void Update() {
    if (!gameActive && Keyboard.current.rKey.wasPressedThisFrame) {
        ResetGame();
    }
}

// Enemy.cs - add as the very first line of Update():
void Update() {
    if (!GameManager.gameActive) return;
```

```
    // ...existing movement and escape code...
}
```



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

Why This Structure

- Static variables and singletons are deliberately NOT used for this: GameManager's shared state stays static from earlier lessons, but the one new reference is an Inspector slot, the simplest and most familiar way to connect two objects. Static members and singletons are a good refactoring topic for Semester 2.
- Tagging the enemy prefab "Enemy" and using FindGameObjectsWithTag() is the standard, idiomatic Unity way to find 'all objects of this kind' - the same category of tool as Godot's Groups, just Unity's own version.

Level 2 / Level 3 Talking Points

- Level 3 bug hunt: kills reduce enemies_remaining but escapes only reduce lives, so a round with some kills and some escapes (for example 3 and 2) ends with no enemies left, no win and no loss, and R does nothing because game_active is still true. This started in Lesson 6 (win = all killed) and Lesson 8 (lose = lives gone). A good fix, and the

one shown here, is that an escape also resolves an enemy and the win check requires lives above 0. Accept any design that guarantees every round ends in exactly one outcome (for example a third 'Round Over' state, or a check that no enemies remain and none are in flight). Test it with 3 kills and 2 escapes, then 4 and 1, then 0 and 5.

- Ask students what would go wrong if the Spawner slot were left empty - ResetGame() would throw a NullReferenceException, a good concrete illustration of why an Inspector reference has to be assigned.
- For students who also did Godot: compare the Inspector reference against Godot's get_tree().get_nodes_in_group() (a built-in group query) and GameMaker's direct object names - three different ways of solving 'reach those other objects,' worth naming explicitly.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Level 3 visual (GameManager.cs OnGUI): bigger, colored end message.
void OnGUI() {
    // ...Level 1 score and lives labels unchanged...
    if (hasWon) {
        GUIStyle bigStyle = new GUIStyle(GUI.skin.label);
        bigStyle.fontSize = 36;
        bigStyle.normal.textColor = Color.yellow;
        GUI.Label(new Rect(250, 120, 500, 60), "You Win! Press R to
Restart", bigStyle);
    }
}

// BUG FIX (one good answer): an escape also resolves an enemy.
GameManager.cs:
public static void RegisterEscape() {
    lives--;
    enemiesRemaining--;
    if (lives <= 0) {
        hasLost = true;
        gameActive = false;
        AudioSource.PlayClipAtPoint(instance.gameOverSound,
Camera.main.transform.position);  // your Lesson 8 Level 3 sound
    } else if (enemiesRemaining <= 0) {
        hasWon = true;
        gameActive = false;
        AudioSource.PlayClipAtPoint(instance.winSound,
Camera.main.transform.position);  // your Lesson 6 Level 3 sound
    } else {
        AudioSource.PlayClipAtPoint(instance.lifeLostSound,
Camera.main.transform.position);  // your Lesson 8 Level 3 sound
    }
}
```

Common Student Mistakes

- Game-state drift: the student added start-up code or a game-over effect beyond the handout (extra green-flag scripts, an animation, a color change, a hidden object) and did not reset it on restart. Symptom: a restart that looks almost right. Walk through the Handout's "Audit Your Game State" list with them.
- Forgetting to tag the enemy prefab "Enemy" - ClearEnemies() silently does nothing, and old enemies pile up after each restart.
- Leaving the Spawner slot on the GameManager component empty, or filling it on the wrong GameObject, so ResetGame() throws a NullReferenceException.
- Placing the gameActive check after movement code in Enemy.cs's Update() instead of as the very first line.